

A Finite Element Solver for Laplace's Equation

Finite Element Methods for Partial Differential Equations

Candidate Number: 1091114

1 Introduction

Finite element methods are useful tools that allow for robust computational analysis of systems that do not admit traditional analytical solutions. As a result, they are used in many fields such as fluid dynamics, electronics, and material stress analysis. They have also given rise to a rich mathematical background that is found at the confluence of linear algebra, functional analysis, and the theory of partial differential equations.

In this special topic report, we aim to explore the mathematical background of these methods in an attempt to further our understanding of their utility, any possible drawbacks to these methods, and how they are addressed. In particular, we shall focus on solving Poisson's equation, given by $-\nabla^2 u = f$ in some two-dimensional domain Ω , with homogeneous Dirichlet boundary conditions on $\partial\Omega$ using finite element methods. We have chosen this project, as we believe that it will provide a good introduction to the numerical aspect of these methods while simultaneously demonstrating as much mathematical rigour as possible to illustrate our understanding.

Then, we shall use our own finite element code to obtain numerical solutions for Poisson's equation across different domains; finding that this can result in increasing difficulty to obtain robust solutions.

Throughout this topic, we have tried to adhere to the notation presented in Farrell [7], and the accompanying lecture slides, for continuity and legibility.

2 Theoretical Motivation

We wish to solve Poisson's equation in two dimensions on a domain of our choice Ω , where we shall impose that this domain is an open, bounded, and connected subset of Euclidean space $\mathbb{R}^3$. We will also impose that our solution, denoted u , must be identically equal to zero on the boundary of our domain, denoted $\partial\Omega$. In other words, we seek to find u , satisfying

$$\begin{aligned} -\nabla^2 u &= f && \text{in } \Omega \\ u &= 0 && \text{on } \partial\Omega. \end{aligned} \tag{1}$$

However, this formulation would require that at every point in the domain, we have $u \in C^2(\Omega)$, $f \in C(\Omega)$ and $u|_{\partial\Omega} \in C(\Omega)$. Unfortunately, we find that the map $-\nabla^2 : C^2(\mathbb{R}) \rightarrow C^0(\mathbb{R})$ is not invertible, as shown in Farrell [7], Gilbarg and Trudinger. [8]. Indeed, this map is neither injective nor surjective. We can easily prove that this map

is not injective: if we take any linear affine transformation of form $(ax + b) \in C^2(\Omega)$, then $-\nabla^2(ax + b) = 0$. This means that the kernel of $-\nabla^2 : C^2(\mathbb{R}) \rightarrow C^0(\mathbb{R})$, denoted $\ker(-\nabla^2) = \{f \in C^2(\mathbb{R}) \mid -\nabla^2 f = 0\}$ is non-trivial, hence our mapping is not invertible.

We stress the importance of this finding, firstly, as it motivates the need for a variational formulation of (1), allowing us to relax our assumptions of the solution u . Secondly, finite element methods are used in order to model physical reality, and discontinuities in solutions are commonplace.

Towards a variational formulation, we note that such weak formulations allow for less regular solutions because multiplication by test functions and Lebesgue integration can ignore the ‘poor behaviour’ of solutions on sets of measure zero. Let us therefore re-formulate our problem to:

$$\int_{\Omega} -(\nabla \cdot \nabla u)v \, dx = \int_{\Omega} f v \, dx. \quad (2)$$

We can now integrate by parts and apply the divergence theorem to the left hand side of (2) and find:

$$\int_{\Omega} \nabla u \cdot \nabla v \, dx - \int_{\partial\Omega} (\nabla u \cdot n)v \, ds = \int_{\Omega} f v \, dx, \quad (3)$$

where n is the unit outwards normal vector of the boundary Ω , and ds is the line element that parameterises $\partial\Omega$. Notice that we have yet to use the boundary condition stated in (1), namely that we know our solution u to be equal to zero on the boundary. This is characteristic of weak formulations, and we can enforce our boundary condition by the properties of the test function v , or more generally, the function space that v occupies (which is yet to be determined, but we shall denote it V). For now, we shall impose the constraint that $v|_{\partial\Omega} = 0$, eliminating the second term on the left hand side of (3). As a result, we can now write down the full weak (or variational) formulation of the Dirichlet boundary value problem for Poisson’s equation:

$$\int_{\Omega} \nabla u \cdot \nabla v \, dx = \int_{\Omega} f v \, dx \quad (4)$$

for all test functions $v \in V$. Since V is an infinite dimensional function space, and we recall finite element methods have been devised as a main way to approach partial differential equations computationally, we must discretise our relation so that it may be implemented. We shall state the *Galerkin approximation* to the weak formulation of the Poisson equation. Our variational problem will discretise to: find $u_h \in V_h$ where

$$\int_{\Omega} \nabla u_h \cdot \nabla v_h \, dx = \int_{\Omega} f v_h \, dx \quad (5)$$

for all $v \in V_h \subset V$, and where $V_h \subset V$ is a finite dimensional subset of the function space V . As our discrete subspace is finite dimensional, let us claim it is of dimension N , then we can write $V_h = \text{span}\{\phi_1, \dots, \phi_N\}$. For brevity, we will also define the *bilinear form* and *linear form*

$$a(u, v) := \int_{\Omega} \nabla u \cdot \nabla v \, dx \quad F(v) := \int_{\Omega} f v \, dx \quad (6)$$

respectively, with $a(u_h, v_h)$ and $F(v_h)$ as their discrete counterparts. We now seek to expand u_h and v_h in terms of these basis functions, by writing

$$v_h = \sum_{i=1}^N V_i \phi_i \quad u_h = \sum_{j=1}^N U_j \phi_j,$$

which allows us to re-write the problem as follows:

$$\begin{aligned} a(u_h, v_h) &= F(v_h) \\ a\left(u_h, \sum_{i=1}^N V_i \phi_i\right) &= F\left(\sum_{i=1}^N V_i \phi_i\right) \\ \sum_{i=1}^N V_i a(u_h, \phi_i) &= \sum_{i=1}^N V_i F(\phi_i). \end{aligned}$$

As this must hold for all values of V_i , this is equivalent to $a(u_h, \phi_i) = F(\phi_i)$ for all $i \in \{1, \dots, N\}$. By a similar substitution for u_h in $a(u_h, v_h)$, we find the linear system

$$AU = b, \quad (7)$$

where $A \in \mathbb{R}^{N \times N}$, and the entries of matrix A are defined by $A_{ji} = a(\phi_i, \phi_j)$, $U = (U_1, \dots, U_N) \in \mathbb{R}^N$ and $b \in \mathbb{R}^N$, with $b_i = F(\phi_i)$. Our computational approach will be centred around solving this linear system. Note, we have not yet determined the function space in which we will proceed.

Definition 1 (Inner product space). Let X be a vector space equipped with an inner product $(\cdot, \cdot) : X \times X \rightarrow \mathbb{R}$, satisfying the following properties:

1. $(u, v) = (v, u)$;
2. $(\alpha u + \beta v, w) = \alpha(u, w) + \beta(v, w)$;
3. $(u, u) \geq 0$, $(u, u) = 0 \iff u = 0$.

An inner product space can induce a norm $\|u\|_X := \sqrt{(u, u)}$.

Definition 2 (Hilbert space). A Hilbert space is a *complete* inner product space. Recall that *completeness* is a property of a normed linear space X means that all Cauchy sequences in X converge in X .

Finally, we shall introduce the *dual* of a Hilbert space.

Definition 3 (Dual space). The *dual* X^* of a Hilbert space X is the space of all bounded linear functionals on X . It has a natural norm induced by the Hilbert space:

$$\|j\|_{X^*} := \sup_{\|v\|_X=1} |j(v)|.$$

We have defined these terms subsequently, as this will now allow us to understand the properties of the function space in which we are looking for our test function v and solution u . In particular, we are looking for a function space V that guarantees the existence of ∇v , the square integrability of the gradient of the function, namely, $\nabla v \in L^2(\Omega)$, and that $v|_{\partial\Omega} = 0$, for every element $v \in V$. This leads us to conclude that V is a Hilbert space (or a Sobolev space by definition) of the form $V = H_0^1(\Omega) := \{v \in H^1(\Omega) \mid v = 0 \text{ on } \partial\Omega\}$. This is the natural space of the problem, which we can illustrate by taking the inner product of u and v with respect to this norm:

$$(u, v)_{H_0^1(\Omega)} = \int_{\Omega} \nabla u \cdot \nabla v \, dx. \quad (8)$$

Similarly for the right hand side of (4), the space $H_0^1(\Omega)$ guarantees that $f \in L^2(\Omega)$, so the expression remains bounded on both sides.

Most importantly, we now turn to ensure that the solution of our variational formulation:

$$\text{find } u \in V \text{ such that } a(u, v) = F(v), \text{ for all } v \in V, \quad (9)$$

is a *well-posed* problem, with a guaranteed existence and uniqueness of a stable solution u . We must introduce two concepts to guarantee this. Firstly, the concept of *boundedness*:

Definition 4 (Boundedness of a bilinear form). A bilinear form $a : V \times V \rightarrow \mathbb{R}$ is said to be *bounded* if there exists $C \in [0, \infty)$ such that $a(u, v) = C\|u\|_V\|v\|_V$, for all $u, v \in V$.

The property of boundedness is shown to be equal to continuity for linear (or in our case, bilinear) operators in normed spaces. A proof of this is available in Kreyszig [10], under ‘Theorem 2.7-9’. We find in Brezis [4] that the bilinear form $a(u, v)$ is bounded, with constant $C = 1$. The next concept we introduce is that of *coercivity*:

Definition 5 (Coercivity of a bilinear form). A bilinear form $a : V \times V \rightarrow \mathbb{R}$ is said to be *coercive on V* or *V -coercive* if there exist an $\alpha > 0$ such that $a(v, v) \geq \alpha \|v\|_V^2$ for all $v \in V$.

From Brezis [4], we can also show that $a(u, v)$ is coercive. We are now in a position to prove that our variational formulation (9) has a unique and stable solution, with the following two theorems.

Theorem 2.1. Let V be a Hilbert space, and suppose that $a : V \times V \rightarrow \mathbb{R}$ is a *symmetric, bilinear form that is continuous on V and coercive on V* . Then, $(V, \|\cdot\|_a)$ is a Hilbert space with the norm $\|\cdot\|_a$ defined by $\|v\|_a = \sqrt{a(v, v)}$.

Proof. We seek to prove that $a(\cdot, \cdot)$ is an inner product on V , and that V is complete, with respect to the induced norm $\|\cdot\|_a$. It is clear from the construction of the bilinear form that $a(v, v) \geq 0$, for all $v \in V$. If $0 = a(v, v) \geq \alpha \|v\|_V^2 \geq 0$ by coercivity, it follows that $v = 0$. Since symmetry and linearity are assumed on $a(\cdot, \cdot)$ (recall our definition of a in (6) to see this), we can conclude that $a(\cdot, \cdot)$ is an inner product on V . Hence, it remains to show that $(V, \|\cdot\|_a)$ is complete. Note that coercivity and continuity guarantee that $\alpha \|v\|_V^2 \leq \|v\|_a^2 \leq C \|v\|_V^2$, for all $v \in V$. This means that these norms are equivalent, and hence by Theorem 2.4-5 of Kreyszig [10], will induce the same notions of convergence and completeness. $\square$

Finally, we can use a special case of the Lax-Milgram theorem, along with the properties of our bilinear and linear forms, which guarantees the existence of a unique stable solution.

Theorem 2.2 (The Lax-Milgram theorem for a symmetric bilinear functional). Let V be a Hilbert space. Let $a : V \times V \rightarrow \mathbb{R}$ be a symmetric bounded, V -coercive bilinear form, and let $F \in V^*$. Consider the variational problem: find $u \in V$ such that $a(u, v) = F(v)$ for all $v \in V$. Then, this problem has a unique and stable solution.

Proof. A proof of this theorem is available in Theorem (2.5.6) in Brenner and Scott [3]. In our case, $\|u\|_a^2 = a(u, u) = \frac{F(u)}{\|u\|_a} \|u\|_a \leq \left(\sup_{v \in V} \frac{F(v)}{\|v\|_a} \right) \|u\|_a = \|F\|_{V^*} \|u\|_a$. Hence, $\|u\|_a \leq \|F\|_{V^*}$. $\square$

Finally, we must discretise this result into its Galerkin approximation. Let us redefine the variational formulation (9), using the notation given in (6) to the discrete form:

$$\text{find } u_h \in V_h \text{ such that } a(u_h, v_h) = F(v_h), \text{ for all } v_h \in V_h \subset V. \quad (10)$$

Indeed, one can show that this discretised approximation is well posed.

Corollary 2.3. Let a and F satisfy the hypothesis of the Lax-Milgram theorem. Then, the Galerkin approximation is well-posed for any closed subspace $V_h \subset V$.

Proof. As $V_h \subset V$, $a : V \times V \rightarrow \mathbb{R}$ is bounded and coercive on V_h , with the same coercivity and boundedness constants. $F \in V_h \rightarrow \mathbb{R}$ is linear and bounded. By assumption, V_h is closed, and hence is a Hilbert subspace of the Hilbert space V . Hence, by the Lax-Milgram theorem, the variational formulation describing the Galerkin approximation is well posed [7]. $\square$

3 The Finite Element Method

We shall start by quoting Céa's Lemma, Farrell [7], which states that the Galerkin approximation to the variational formulation is a *quasi-optimal* solution, which satisfies $\|u - u_h\|_V \leq \frac{C}{\alpha} \max_{v_h \in V} \|u - v_h\|$, where α and C satisfy the same coercivity and continuity constants defined previously (in our case, as our bilinear form $a : V \times V \rightarrow \mathbb{R}$ is symmetric, it can be shown to adhere to a tighter constant constraint, equal to $\sqrt{\frac{C}{\alpha}}$ [3]).

Our next step is to construct our discrete subspace $V_h \subset V$ with a good approximating property on generic domains. To do this, we shall illustrate the finite element method, and provide a worked example of each step we have taken, in order to solve our problem.

Definition 6 (Finite element). A *finite element* is the triple $(K, \mathcal{V}, \mathcal{L})$ where

1. The cell K is a bounded, closed subset of $\mathbb{R}^n$ with a non-empty connected interior and a smooth boundary;
2. The space $\mathcal{V} = \mathcal{V}(K)$ is a finite-dimensional function space on K of dimension d , that is, $\dim(K) = d < \infty$;
3. The set of degrees of freedom $\mathcal{L} = \{\ell_1, \dots, \ell_d\}$ is a basis for $\mathcal{V}^*$, the dual space of $\mathcal{V}$.

In our case, we have chosen to use linear Lagrange finite elements of a triangular shape, so we have taken $(K, \mathcal{V}, \mathcal{L})$, where $K = \triangle$, $\mathcal{V} = \text{span}\{1, x_1, x_2\}$, $\mathcal{L} = \{\ell_1, \ell_2, \ell_3\}$, $\ell_i : v \mapsto v(x_i)$. An example of a linear Lagrange triangular finite element is available in Figure 8 of Appendix A. Once we have defined our finite elements, we can interpolate functions onto them, which we define below:

Definition 7 (Interpolant of an element). Let $(K, \mathcal{V}, \mathcal{L})$ be a finite element. For a suitable function space H , define the *interpolant* $\mathcal{I}_k : H \rightarrow \mathcal{V}$ by $\mathcal{I}_k : u \mapsto \mathcal{I}_k u$, such that

$$\ell_i(\mathcal{I}_k u) = \ell_i(u), \text{ for all } \ell_i \in \mathcal{L}, i = 1, \dots, d.$$

In other words, the interpolant matches the value of the function at the degrees of freedom.

Next, we must select a basis for the space $\mathcal{V}$, in particular, we shall use a nodal basis.

Definition 8 (Nodal basis). A *nodal basis* is a basis $\{\phi_1, \dots, \phi_d\}$ of $\mathcal{V}$ with the property that $\ell_i(\phi_j) = \delta_{ij}$, for $i, j = 1, \dots, d$, and where δ_{ij} is the Kronecker delta function.

Note the subtlety that we had defined $\mathcal{L}$ to be a basis for the dual space $\mathcal{V}^*$ of $\mathcal{V}$. One may ask whether it necessarily follows that $\mathcal{V}$ is the dual of $\mathcal{L}$? By Remark 2.3.2 in Ern and Guermond [5], we see that $\mathcal{V} \cong \mathcal{L}^*$ and $\mathcal{L} \cong \mathcal{V}^*$, also known as a *dual pair*.

For example, in our case of a linear Lagrange interpolant in a triangular domain, letting K be a triangle with vertices at $(0,0)$, $(1,0)$ and $(0,1)$. With $\mathcal{V}$ given by $\text{span}\{1, x_1, x_2\}$ and $\mathcal{L}$ being the pointwise evaluation at the vertices, then the nodal basis for $x = (x_1, x_2)$ is given by

$$\phi_1(x) = 1 - x_1 - x_2, \quad \phi_2(x) = x_1, \quad \phi_3(x) = x_2.$$

Since we see that $\ell_1(v) = v(0,0)$, $\ell_2(v) = v(1,0)$, and $\ell_3(v) = v(0,1)$, we see that the relation $\ell_i(\phi_j) = \delta_{ij}$ holds. Next, we wish to interpolate functions onto our basis. Indeed for a nodal basis, we have a particularly simple expression for the nodal basis (one can simply apply the definition of the nodal basis to our interpolant mapping):

$$\mathcal{I}_K u(x) = \sum_{i=1}^d \ell_i(u) \phi_i(x), \text{ for } x \in K. \quad (11)$$

We can now define a global basis for our function space, given by

$$V_h = \text{span}\{\phi_1, \dots, \phi_N\} \subset V, \quad (12)$$

so that we can decompose our original domain Ω into cells on which we can define our finite elements. Let us therefore define a mesh.

Definition 9 (Mesh). A *mesh* $\mathcal{M}$ is a geometric decomposition of domain Ω into a finite set of *cells* $\mathcal{M} = \{K_i\}$ such that

1. $\cup_i K_i = \bar{\Omega}$ (the union of all cells K_i spans the closure of the domain);
2. $K_i \cap K_j$ for $i \neq j$ is exactly one point, it is a common vertex of both K_i and K_j ;
3. If $K_i \cap K_j$ for $i \neq j$ is not empty or one point, it must be a common facet of both K_i and K_j (this amounts to an edge in two dimensions).

Note that in our case, we had used a mesh generation algorithm by Persson and Strang [12], which had been written in MATLAB. The code was then converted into Python, onto which the basis was superposed. From here, we equip each cell $K \in \mathcal{M}$ with a finite element, so we have a set of finite elements $\{(K, \mathcal{V}_K, \mathcal{L}_K) : v|_K \in \mathcal{V}_K, \forall K \in \mathcal{M}\}$. As a result, we can now define V_h , our discretisation of the function space V in which our continuous variational problem exists:

$$V_h := \{v \in V : v|_K \in \mathcal{V}_K, \forall K \in \mathcal{M}\}. \quad (13)$$

In our case, we had previously concluded that $V = H_0^1(\Omega)$, the natural space to define Poisson's equation with homogeneous Dirichlet boundary conditions. It follows that we must ensure the existence and continuity of the functions at the boundaries between finite elements. We seek to match the degrees of freedom (in our case, the vertices of the triangular elements) with neighbouring elements. This is known as a *local-to-global mapping*.

Definition 10 (Local-to-global mapping). For each cell $K \in \mathcal{M}$ we specify the *local-to-global map*

$$\iota_K : \{1, \dots, d(K)\} \rightarrow \{1, \dots, N\}$$

which relates the *local* degrees of freedom $\ell_i^K(v)$ to the *global* degrees of freedom. We can then define the *local-to-global map*

$$\ell_{\iota_K(i)}(v) = \ell_i^K(v|_K), \text{ for } i = \{1, \dots, d(K)\}.$$

In other words, we can say that the map defines “the i^{th} local degree of freedom on cell K to the function v , restricted to K ”.

The result of this map is that if two different degrees of freedom, say ℓ_i^K and $\ell_j^{K'}$, for two different cells K and K' , map to the same global degree of freedom, we must have that

$$\ell_i^K(v|_K) = \ell_j^{K'}(v|_{K'}),$$

guaranteeing that the degrees of freedom of the two cells at the common edge or vertex agree. It is important to remember that our natural space was $V := H_0^1(\Omega)$. One may ask whether the derivatives of $v_h \in V_h$ should also be continuous across our local-to-global mapping boundaries? We find that this is not necessary, as described by Theorem 2.2.3 in Ern and Guermond [5]. This states that for our functions $v_h \in V_h$, we have that $v_h \in C^0(\Omega)$, that is to say they are continuous by the local-to-global mapping, and that their piecewise-defined derivatives lie in $L^2(K)$. In other words, the weak derivatives ∇v_h exist *locally* in each element $L^2(K)$. As a result, we find that $V_h \subset H^1(\Omega)$. This shows that our local-to-global map is *conforming*.

Definition 11 (Conforming approximation). Suppose the continuous variational problem is well-posed over a Hilbert space V . If $V_h \subset V$, the approximation is *conforming*; if $V_h \not\subset V$, then the approximation is said to be *nonconforming*.

We are now in a position to define our global function space. We can take our local-to-global map and gather the degrees of freedom we had defined:

$$\mathcal{L} = \{\ell_1, \dots, \ell_N\} = \bigcup_{K \in \mathcal{M}} \{\ell_{i_{K(i)}}, \text{ for } i = 1, \dots, d(K)\},$$

and use the associated nodal basis as the basis for V_h :

$$V_h = \text{span}\{\phi_1, \dots, \phi_N\}, \quad l_i(\phi_j) = \delta_{ij}, \quad \phi_i|_K \in \mathcal{V}_K, \quad \forall K \in \mathcal{M}.$$

From here, we can build a global interpolation operator:

Definition 12 (Global interpolation operator). Let V_h be a finite element function space found by equipping a mesh $\mathcal{M}$ with finite elements. Then the global interpolation operator $\mathcal{I}_h : H \subset V \rightarrow V_h$ is defined by $(\mathcal{I}_h u)|_K = \mathcal{I}_K u$ and $\mathcal{I}_h u$ satisfies any continuity requirements.

That is, the global interpolation operator is defined element-wise via the local interpolants $\mathcal{I}_K$, and ensures global continuity by construction. In our case, as we are using linear Lagrange interpolants in two dimensions, we can employ the Bramble-Hilbert lemma as a known result in interpolation theory (see Mandel [11]), which leads us to conclude that $V = H^1(\Omega)$ and $H = H^2(\Omega)$.

We will explore the algorithm for computing A and b for the linear system defined in (7). Firstly, we will focus on computing the *local stiffness matrix*, which we shall denote A_K . Computing the local stiffness matrix for each cell in the mesh is a more

efficient method to find the global stiffness matrix A , compared to calculating each $A_{ji} = a(\phi_i, \phi_j)$, as we know from previous work that all basis functions have compact support, so for most pairs i, j , $a(\phi_i, \phi_j) = 0$. Once we have computed A_K , for all K , we can then add them together in order to find A , as in Figure 1 below.

```

K = lil_matrix((N,N), dtype=float)  # Sparse global matrix

...

for tri_nodes in t:
    ...
    Ke = np.zeros((3,3))              # Local stiffness matrix
    ...
    for i_local, i_global in enumerate(tri_nodes):
        rhs[i_global] += Fe[i_local]
        for j_local, j_global in enumerate(tri_nodes):
            K[i_global, j_global] += Ke[i_local, j_local]

```

Figure 1: A truncated section of the code we have written, illustrating the computing of the local stiffness matrix, followed by its addition into the global stiffness matrix, for each local matrix.

Next, we shall describe how to compute the local stiffness matrix along with the *local load vector*, the entries of which are the given by $\int_K f \phi_j dx$, and is the right-hand side of our expression on cell K . It is efficient to introduce a *reference element* $(\hat{K}, \hat{\mathcal{V}}, \hat{\mathcal{L}})$ and a set of *diffeomorphisms* (an invertible function or isomorphism that maps one differentiable manifold, in our case, $\hat{K}$, to another such that both the function and its inverse are continuously differentiable) $\{F_K : K \in \mathcal{M}\}$ such that $K = F_K(\hat{K})$, for all $K \in \mathcal{M}$. In this case, F_K is an affine map, which means it is of the form of a shifted linear system, $F_K = J_K \hat{x} + c$, where c is a vector of size d , as shown in Figure 2. As this map is invertible by construction, the map F_K can generate a function space on K via

$$\mathcal{V}(K) = \{v = \hat{v} \circ F_K^{-1} : \hat{v} \in \hat{\mathcal{V}}\}, \quad (14)$$

and a set of degrees of freedom defined on K via

$$\mathcal{L}(K) = \{\ell(v) = \hat{\ell}(\hat{v}) = \hat{\ell}(v \circ F_K) : \hat{\ell} \in \hat{\mathcal{L}}\}. \quad (15)$$

As a result, we have obtained a nodal basis for $\mathcal{V}(K)$ from one on $\hat{\mathcal{V}}$. Let us now suppose that $\{\hat{\phi}_i\}_{i=1}^d$ satisfies the nodal basis expression, namely $\hat{\ell}_i(\hat{\phi}_j) = \delta_{ij}$. Let

us further define $\phi_i^K := \hat{\phi}_i \circ F_K^{-1}$. Then, using the formulations in (14) and (15) respectively,

$$\ell_j^K(\phi_i^K) = \hat{\ell}_j(\phi_i^K \circ F_K) = \hat{\ell}_j(\hat{\phi}_i \circ F_K^{-1} \circ F_K) = \hat{\ell}_j(\hat{\phi}_i) = \delta_{ij}. \quad (16)$$

Therefore, in order for mapping to a reference element to work, we must have that $\mathcal{V}(K) = \mathcal{V}_K$ and $\mathcal{L}(K) = \mathcal{L}_K$, such that the finite element constructed via transformation from the reference element is exactly the same as the finite element constructed directly onto cell K .

For example, as we wish to solve Poisson's equation, we need to solve

$$\int_K \nabla_x \phi_i(x) \cdot \nabla_x \phi_j(x) \, dx. \quad (17)$$

We can transform this into our reference space by using the Jacobian of the transformation:

$$\int_K \nabla_x \phi_i(x) \cdot \nabla_x \phi_j(x) \, dx = \int_{\hat{K}} \nabla_{\hat{x}} \hat{\phi}_i(\hat{x}) \cdot \nabla_{\hat{x}} \hat{\phi}_j(\hat{x}) |J_K(\hat{x})| \, d\hat{x}, \quad (18)$$

where the Jacobian is given by $[J_K(\hat{x})]_{ij} = \frac{\partial x_i}{\partial \hat{x}_j}$. Note, however, that our gradient operator ∇_x is still applying derivatives with respect to the initial physical co-ordinates, x , not the transformed co-ordinates of our reference element, $\hat{x}$. As a result, we shall apply the chain rule, writing

$$\frac{\partial \phi}{\partial x_k} = \sum_{l=1}^n \frac{\partial \phi}{\partial \hat{x}_l} \frac{\partial \hat{x}_l}{\partial x_k}, \text{ for } k = 1, \dots, n.$$

We find that $\nabla_x \hat{\phi}(\hat{x}) = J_K^{-T}(\hat{x}) \nabla_{\hat{x}} \hat{\phi}(\hat{x})$ and hence we can write

$$\int_K \nabla_x \phi_i(x) \cdot \nabla_x \phi_j(x) \, dx = \int_{\hat{K}} \left(J_K^{-T}(\hat{x}) \nabla_{\hat{x}} \hat{\phi}_i(\hat{x}) \right) \cdot \left(J_K^{-T}(\hat{x}) \nabla_{\hat{x}} \hat{\phi}_j(\hat{x}) \right) |J_K(\hat{x})| \, d\hat{x}, \quad (19)$$

which we can solve numerically or via software. In our case, we used the quadrature rule. By a similar calculation, we can transform the local load vector:

$$\int_K f(x) \phi_j(x) \, dx = \int_{\hat{K}} f(F_K(\hat{x})) \hat{\phi}_j(\hat{x}) |J_K(\hat{x})| \, d\hat{x}. \quad (20)$$

Below, Figure 2 shows us this code snippet where a computational version of the method we have described, where $\mathbf{r}$ and $\mathbf{s}$ are the (barycentric) co-ordinates of the reference triangle, and $\mathbf{xq}[0]$ and $\mathbf{xq}[1]$ are the physical co-ordinates.

```

# Vertices of triangle
A = p[tri[0]]
B = p[tri[1]]
C = p[tri[2]]

# Area of triangle
area = 0.5 * abs((B[0]-A[0])*(C[1]-A[1]) - (B[1]-A[1])*(C[0]-A[0]))

# Gradients of linear basis functions approximations
grad = [np.array([(B[1]-C[1]), (C[0]-B[0])]) / (2*area),
         np.array([(C[1]-A[1]), (A[0]-C[0])]) / (2*area),
         np.array([(A[1]-B[1]), (B[0]-A[0])]) / (2*area)]

# Local stiffness matrix (3x3)
Ke = np.array([[np.dot(gi, gj) * 2 * area for gj in grad]
               for gi in grad])
# Local load vector using barycentric quadrature
quad_pts = np.array([[1/6, 1/6], [2/3, 1/6], [1/6, 2/3]])
weights = np.array([1/6, 1/6, 1/6])
Fe = np.zeros(3)
J = np.column_stack((B - A, C - A))
for (r, s), w in zip(quad_pts, weights):
    phi = np.array([1 - r - s, r, s])
    x_q = A + J @ [r, s]
    fq = f(x_q[0], x_q[1])
    detJ = abs(np.linalg.det(J))
    Fe += phi * fq * w * detJ

```

Figure 2: A truncated section of the code written which shows the definition and numerical integration by quadrature of the reference element using barycentric coordinates r and s , in order to calculate the local stiffness matrix $\mathbf{K}_e$ as well as the local load vector $\mathbf{F}_e$.

4 Results

In this section, we aim to solve $-\nabla^2 u = f$ with homogeneous Dirichlet boundary conditions via finite element methods in three distinct domains, in order to test the robustness of our method. The domains we have chosen are:

- (i) A unit square domain;

- (ii) A circle of unit radius;
- (iii) A non-convex domain.

We have chosen these three domains in order to explore three cases where we postulate that our results of convergence and error estimations may differ. Firstly, a square domain admits an analytical expression for a solution, as does a circular domain. However, a square domain can be easily triangulated by our mesh generator, whereas a circle is *nonpolytopic*, meaning it contains curved surfaces, and therefore it can only be approximated by *polytopic* (composed of flat surfaces or edges) polygons, such as triangles. Finally, we have chosen two *non-convex* domains; an ‘L-shaped’ domain, and a circular domain with a quarter-segment removed. We will find the re-entrant vertex will significantly impact our error analysis, forcing us to adjust our method, as well as allowing for some theoretical exploration on problems in non-convex domains. In order to derive expressions with which we can bound our error, we must define the *mesh size* and *shape-regularity* of our mesh.

Definition 13 (Mesh size). Given a mesh $\mathcal{M}$, we can define the mesh size, h , given by $h = \max_{K \in \mathcal{M}} \text{diam}(K)$, where we have defined $\text{diam}(K) = \max\{\|x_1 - x_2\| : x_1, x_2 \in K\}$, the diameter of the cell. In our case, this definition resolves to the longest edge of the largest triangle in our mesh.

We must also introduce the concept of shape-regularity of our sequence of meshes $(\mathcal{M}_h)_h$, as we reduce the mesh size of our elements in order to converge onto an answer.

Definition 14 (Shape-regularity of a mesh sequence). A mesh sequence is *shape-regular* if there exists a constant σ , independent of h , such that

$$\sup_{h>0} \max_{K \in \mathcal{M}_h} \frac{h_K}{\rho_K} \leq \sigma, \quad (21)$$

where we have defined ρ_K as the *incircle diameter* of cell K , the diameter of the largest circle that can be entirely contained in cell K .

Finally, we will introduce the Sobolev *seminorm* of Sobolev space $W_p^k(\Omega)$ (recalling that our subspace $H^1(\Omega)$ is a Sobolev space by definition), which will allow us to bound the interpolation error of our computed solution. We have that

$$|u|_{W_p^k(\Omega)} := \left(\sum_{|\alpha|=k} \|D^\alpha u\|_{L^p(\Omega)}^p \right)^{\frac{1}{p}}, \quad (22)$$

where D^α is the differential operator for all derivatives of cumulative order α . This is not a strict norm because $|u|_{W_p^k(\Omega)} = 0 \not\Rightarrow u = 0$ in general. We are finally in a position to bound the interpolation error, by the following theorem, stated in Farrell [7].

Theorem 4.1. Let $(V_h)_h$ be the function space constructed with continuous Lagrange elements of order p on a shape-regular sequence of meshes $(\mathcal{M}_h)_h$ indexed by mesh size h . Let $u \in H^{p+1}(\Omega)$ with $p + 1 > \frac{d}{2}$, and suppose that $\mathcal{I}_h : H^{p+1}(\Omega) \rightarrow V_h$ is the interpolation operator associated with V_h . Then, there exist constants $D, D' < \infty$, independent of u such that

$$\|u - \mathcal{I}_h u\|_{H^1(\Omega)} = Dh^p |u|_{H^{p+1}(\Omega)}. \quad (23)$$

and

$$\|u - \mathcal{I}_h u\|_{L^2(\Omega)} = D'h^{p+1} |u|_{H^{p+1}(\Omega)} \quad (24)$$

Proof. A proof of this theorem is available in Chapter 4.4 of Brenner and Scott [3]. $\square$

In our case, since we had used linear Lagrange elements, we have that $p = 1$. Therefore, we can state that if $u \in H^2(\Omega)$, the error between the exact solution and our interpolated solution scales like $\mathcal{O}(h)$. In order to show that $u \in H^2(\Omega)$, we may use the following elliptic regularity result.

Theorem 4.2. Let Ω be a *convex* domain. Suppose that $f \in L^2(\Omega)$. Then, the solution $u \in H_0^1(\Omega)$ of Poisson's equation is an element of $H^2(\Omega) \cap H_0^1(\Omega)$, and satisfies

$$|u|_{H^2(\Omega)} \leq c \|f\|_{L^2(\Omega)}, \quad (25)$$

for some constant c independent of u and f .

Proof. A proof of this result is available in Evans [6], followed by remarking that since $u \in H_0^1(\Omega)$, we may use Poincaré's inequality, to show that $|\cdot|_{H^2(\Omega)}$ and $\|\cdot\|_{H^2(\Omega)}$ are equivalent up to a constant in this special case. $\square$

Lastly, we must employ the *Aubin-Nitsche duality argument*, available in Farrell [7] in order to re-express $\|u - \mathcal{I}_h u\|_{H^1(\Omega)}$ and $\|u - \mathcal{I}_h u\|_{L^2(\Omega)}$ in terms of estimates of $\|u - u_h\|_{H^1(\Omega)}$ and $\|u - u_h\|_{L^2(\Omega)}$. Upon doing so, we find

$$\|u - u_h\|_{H^1(\Omega)} \leq C_1 h \|f\|_{L^2(\Omega)} \quad (26)$$

and

$$\|u - u_h\|_{L^2(\Omega)} \leq C_2 h^2 |u|_{H^1(\Omega)}, \quad (27)$$

for some constants C_1 and C_2 .

Case (i): A unit square domain

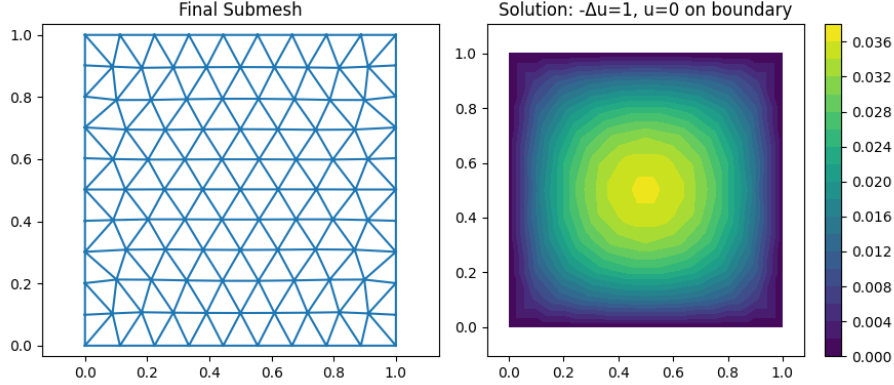


Figure 3: A mesh of mesh size $h = 0.1$ generated on the unit square domain (left), along with the numerical solution of Poisson’s equation $-\nabla^2 u = 1$ with homogeneous Dirichlet boundary conditions, found using a finite element method solver written in Python.

As we have a unit square, we can construct the analytical solution for Poisson’s equation in this domain, given by [9]:

$$u(x) = \sum_{\substack{m=1 \\ m \text{ odd}}}^{\infty} \sum_{\substack{n=1 \\ n \text{ odd}}}^{\infty} \frac{16}{\pi^4} \frac{1}{mn(m^2 + n^2)} \sin(m\pi x_1) \sin(n\pi x_2). \quad (28)$$

This means that we can calculate the error between our computed solution and the analytical solution easily. This is useful when developing finite element software as we had done, since it allows for the comparison between the exact analytical solution and the computed solution at every iterate, as given by the left hand sides of expressions (26) and (27). We can then compare them with the right hand sides of their respective equations, in order to ensure that our code is working as intended.

In Figure 4, we see excellent agreement with our theoretical error convergence bounds (26) and (27). Since we had a particularly simple domain, a uniform grid was constructed of squares of side length h , and then each square cell was split into two in order to recover the triangular linear Lagrange elements on which we would set up our finite element solver¹. This was done for simplicity, in order to make sure there would be minimal geometric errors arising from poor meshing procedures, although this was not a major source of inaccuracy. We began by considering a coarse mesh size $h = 0.2$,

¹Note, this is not the procedure we see in Figure 3, where a mesh generating software [12] was used in general, but a uniform grid was favoured for error analysis where applicable.

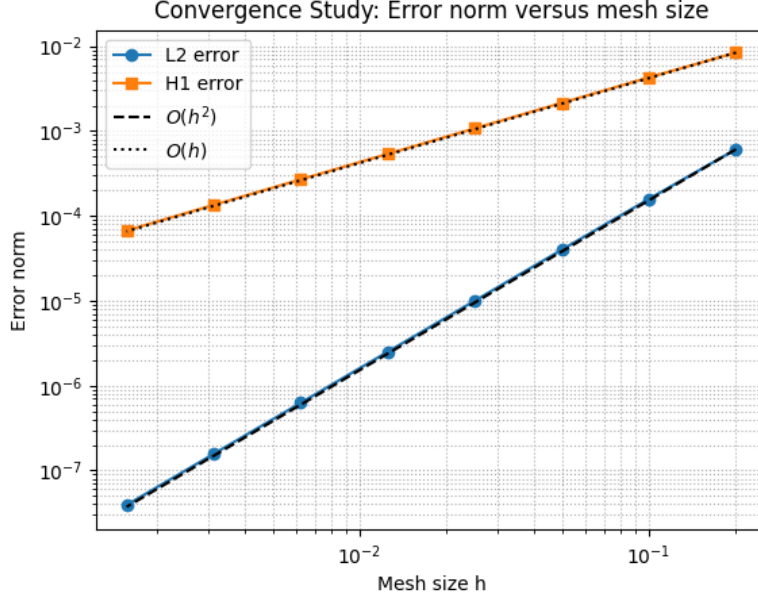


Figure 4: A log-log plot of the $L^2(\Omega)$ and $H^1(\Omega)$ error bounds for Poisson's equation $-\nabla^2 u = 1$ with homogeneous Dirichlet boundary conditions on a unit square. Overlaid are the dashed and dotted lines showing $\mathcal{O}(h^2)$ and $\mathcal{O}(h)$ convergence respectively with mesh size.

before implementing the finite element algorithm outlined previously and solving the resulting linear system. Then, we would calculate the resulting errors in the $L^2(\Omega)$ and $H^2(\Omega)$ norms. Next, the mesh size would be halved, before the procedure was repeated, and the calculated bound was plotted onto Figure 4, resulting in values for $\|u - u_h\|_{L^2(\Omega)}$ and $\|u - u_h\|_{H^1(\Omega)}$ for $h = 0.2 \times 2^{-k}$, $k = 0, 1, \dots, 7$, terminating at $h = 1.5625 \times 10^{-3}$. We had chosen these values as one can easily visually determine that as our mesh size halves, so does our $H^1(\Omega)$ norm error, and our $L^2(\Omega)$ norm error decreases by a factor of four.

Case (ii): A unit circular domain

In this section, our main objective is to compare the error convergences of our two convex domains, with the distinction that a circular domain is nonpolytopic, but can still admit an analytical expression. In this case, we have chosen the right hand side of (1) to be $f = -4 + 16r^2$. As a result, we find the exact solution u to be

$$u(r) = r^2(1 - r^2), \quad (29)$$

where we r is the radial polar co-ordinate. We see from Figure 5 that the mesh

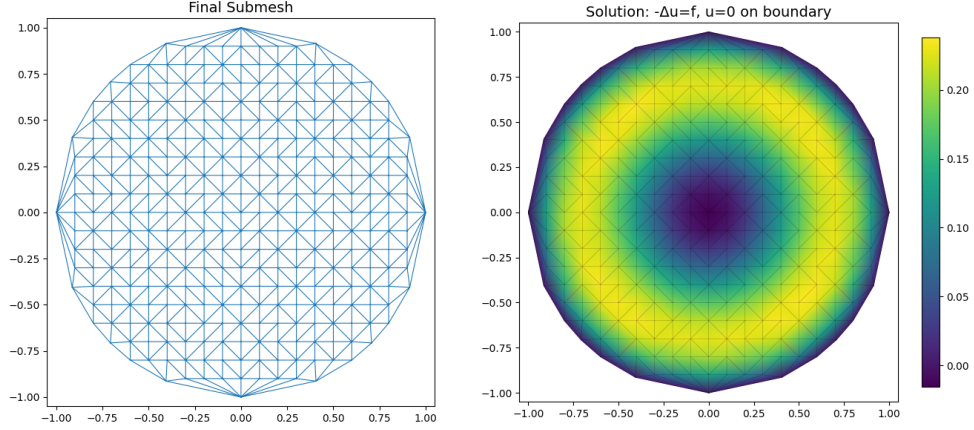


Figure 5: A generated mesh and numerical solution for Poisson’s equation $-\nabla^2 u = f$ on a circular domain of radius 1 with homogeneous Dirichlet boundary conditions. We see that a mesh where each cell is of approximately uniform size and shape is much more difficult as our domain is nonpolytopic.

generator now struggles to fit cells of comparable area and shape to the domain, as well as having to approximate the curved surface at the boundary (we can recall that our theorem pertaining to the convergence of our solutions depends implicitly on the shape-regularity of the domain in which the finite element method is being used). We also note the inherent approximations arising from the transformations from our Cartesian co-ordinate system used by our mesh generator to the polar co-ordinate system we had used to define f , and similarly, the Laplacian operator.

As in the case of the unit square domain, we had used a sequence of mesh sizes given by $h = 0.2 \times 2^{-k}$, $k = 0, 1, \dots, 7$, terminating at $h = 1.5625 \times 10^{-3}$. However, we see in Figure 6 an inaccuracy of convergence for both the $H^1(\Omega)$ norm and the $L^2(\Omega)$ norm. Focusing first on the $L^2(\Omega)$ norm, we see that a slope of 1.62 is far away from the $\mathcal{O}(h^2)$ convergence we would usually expect. Since our code had remained unchanged from the square domain calculations, we conclude that the main source of inaccuracy is the geometric approximations of the mesh generation. Indeed, we see from Theorems 1 and 3 from Bramble, Dupont, and Thomée [2], that naïvely applying our method results in an error convergence closer to $\mathcal{O}(h^{\frac{3}{2}})$.

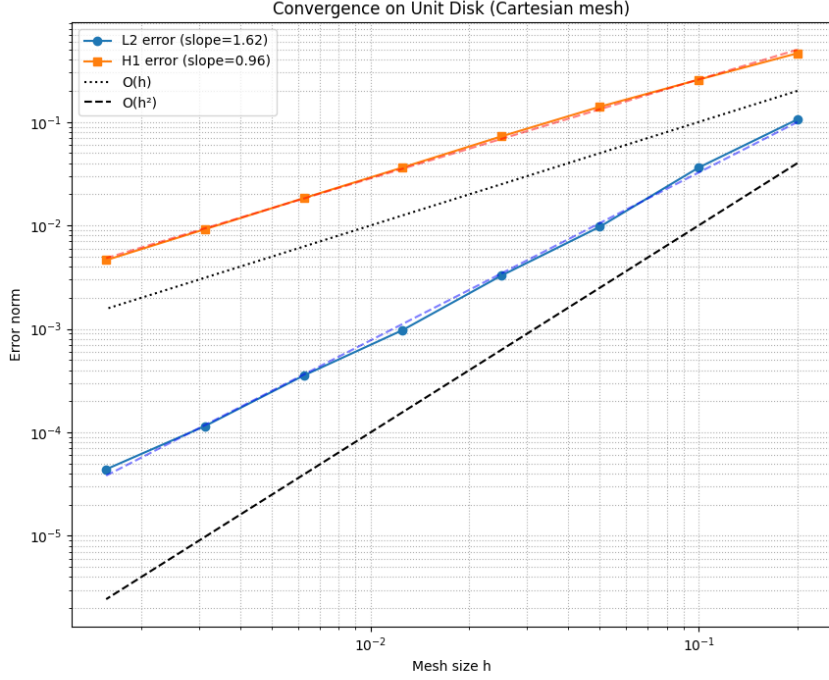


Figure 6: A log-log plot of the $L^2(\Omega)$ and $H^2(\Omega)$ error bounds for Poisson's equation in a circular domain $-\Delta u = -4 + 16r^2$, along with the average gradient of the error convergence and dotted lines showing $\mathcal{O}(h)$ and $\mathcal{O}(h^2)$ mesh size convergence for comparison.

In the case of the $H^1(\Omega)$ error norm, we see this norm reaches an average slope of 0.96, a good estimation to the $\mathcal{O}(h)$ convergence we expect. One may ask why, if the $L^2(\Omega)$ norm is so severely affected, and is included in the calculation of the $H^1(\Omega)$ norm, why do we not expect a similar constraint on this norm? We find that usually, the gradient term of $\|\cdot\|_{H^1(\Omega)}$ dominates. This is particularly important in our case, since we had employed linear Lagrange elements, meaning our gradients are constant functions, and therefore they are not significantly degraded around the boundaries due to our approximations.

Case (iii): A non-convex domain

In this section, we shall consider two non-convex domains. This will mean that our assumptions used in Theorem 4.2 shall no longer hold, which will give rise to less predictable results but richer theories of convergence. To illustrate this, we shall consider

an ‘L-shaped’ domain, taken by using the unit square domain we had considered in Case (i) and removing one quarter of the domain in order to produce an L-shape, the mesh and convergence plot of which are available in Appendix B, and though we find a reasonable mesh generation, our convergence plot is unsatisfactory. Secondly, we shall take the circle of unit radius used in Case (ii), and similarly remove one-quarter segment of the area and form another non-convex domain, resembling three-quarters of a circular domain. For the case of the L-shaped domain, we shall keep all other aspects of the algorithm the same, and for the three-quarter circular domain, we shall instead use a polar mesh.

Firstly, we have marked the $\mathcal{O}(h^{\frac{1}{3}})$ and $\mathcal{O}(h^{\frac{2}{3}})$ as the expected convergence rates for the $H^1(\Omega)$ and $L^2(\Omega)$ norms respectively. We see from Section 4 of Bacuta, Bramble, and Xu [1], that if we take the largest internal angle of our convex domain to be $\frac{3\pi}{2}$, then by Theorem 4.1 of [1], we find $\mathcal{O}(h^{\frac{2}{3}})$ convergence in $\|\cdot\|_{L^2(\Omega)}$ and hence $\mathcal{O}(h^{\frac{1}{3}})$ in $\|\cdot\|_{H^1(\Omega)}$.

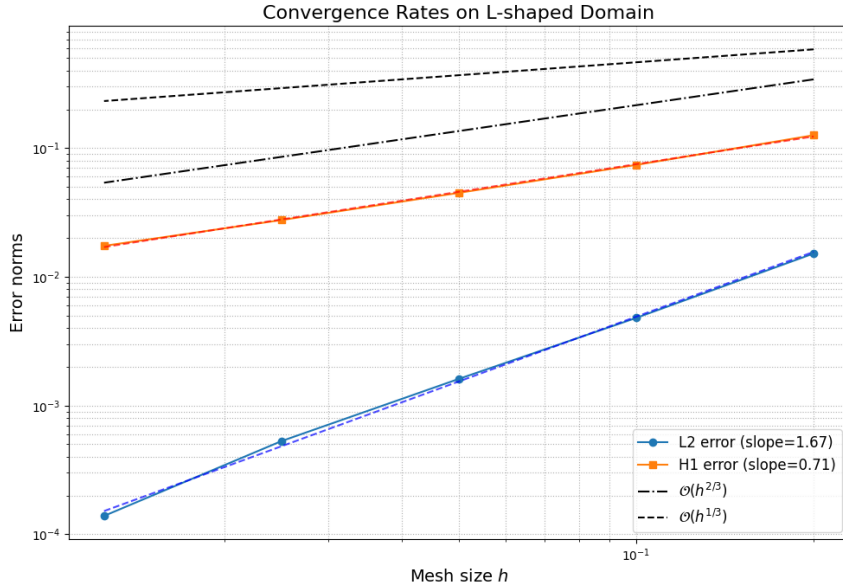


Figure 7: A plot showing the convergence of the finite element method in the three-quarter circular domain, solving Poisson’s equation $-\Delta u = f$, where $f = 0$ in this case (Laplace’s equation), and with homogeneous boundary conditions. We have chosen a homogeneous form because it admits a simple exact solution, with a singularity at the point $r = 0$, making it challenging for numerical schemes to approximate.

In the case of Figure 7, as we have used a polar mesh which radiates outwards, we find a better than anticipated convergence plot. This is primarily as a consequence

of two changes that we had implemented in our algorithm, which helped improve the convergence of the numerical solution to the exact solution. Firstly, we see from Appendix C that our cells become increasingly concentrated around the singular point, so that we can better approximate gradient in terms of our piece-wise elements. Secondly (and perhaps more importantly in this case), we have defined `r_min = 1e-3` as the smallest distance from the singularity we allow our domain to reach. Indeed as we reduce this parameter, our convergence also decreases. This is because our system admits the solution $u(r, \theta) = r^{\frac{2}{3}} \sin(\frac{2\theta}{3})$, although we have a singularity at the origin, we find the solution has good convergence properties on the rest of the domain, and we have $u \in H^2(\Omega)$ everywhere else.

5 Conclusion

This special topic report has given an overview of the finite element method, with particular reference to finding numerical solutions to Poisson’s equation in a variety of domains, using linear Lagrange interpolants on a triangular cell with a nodal basis. We began by introducing the broad motivations of the finite element method, and demonstrated how it can ultimately be used to transform a partial differential equation into a linear system. We have also demonstrated its relative simplicity to implement, by choosing to develop our own code rather than a readily available software package. We found that even a rudimentary algorithm can be used effectively to solve partial differential equations which could quickly approach an exact solution with decreasing mesh size. We had also briefly explored the drawbacks of such regimes, such as in the cases of nonpolytopic or non-convex domains, where a more nuanced understanding of convergence theorems was needed in order to obtain adequate convergence. This is where we believe finite element softwares would have been much more accurate. There are many directions our future research may take, we have neglected entirely the mathematics of choosing optimal meshes for different domains, we had not considered non-linear problems or indeed homogeneous and inhomogeneous Neumann or Robin boundary conditions more broadly. We also found in our research, and algorithm in Bramble, Dupont, and Thomée [2] that admits $\mathcal{O}(h^4)$ convergence in the $L^2(\Omega)$ norm through the use of domain approximation techniques and cubic spline interpolation, something we would be most interested in exploring in the future.

References

- [1] C. Bacuta, J.H. Bramble, and J. Xu. “Regularity estimates for elliptic boundary value problems with smooth data on polygonal domains.” In: *Journal of Numerical Mathematics* (2002).
- [2] J.H. Bramble, T. Dupont, and Vidar Thomée. “Projection methods for Dirichlet’s problem in approximating polygonal domains with boundary-value corrections”. In: *American Mathematical Society* (1972).
- [3] S.C. Brenner and L.R. Scott. *The Mathematical Theory of Finite Element Methods*. Springer, 2008.
- [4] H. Brezis. *Functional Analysis, Sobolev Spaces and Partial Differential Equations*. Springer, 2011.
- [5] A. Ern and J.-L. Guermond. *Theory and Practice of Finite Elements*. Springer, 2004.
- [6] L.C. Evans. *Partial Differential Equations*. American Mathematical Society, 2010.
- [7] P. E. Farrell. *C6.4 Finite Element Methods for Partial Differential Equations*. Univeristy of Oxford, 2025.
- [8] D. Gilbarg and N. S. Trudinger. *Elliptic Partial Differential Equations of Second Order. Third Edition*. Springer, 2001.
- [9] E. Kreyszig. *Advanced Engineering Mathematics*. Wiley, 2011.
- [10] E. Kreyszig. *Introductory Functional Analysis with Applications*. New York, Wiley, 1978.
- [11] Jan Mandel. *The Bramble-Hilbert Lemma*. 2007. arXiv: 0710.5148 [math.NA]. URL: <https://arxiv.org/abs/0710.5148>.
- [12] P.-O. Persson and Gilbert Strang. “A Simple Mesh Generator in MATLAB”. In: *Society for Industrial and Applied Mathematics* (2004).

Appendix A: A finite element reference triangle

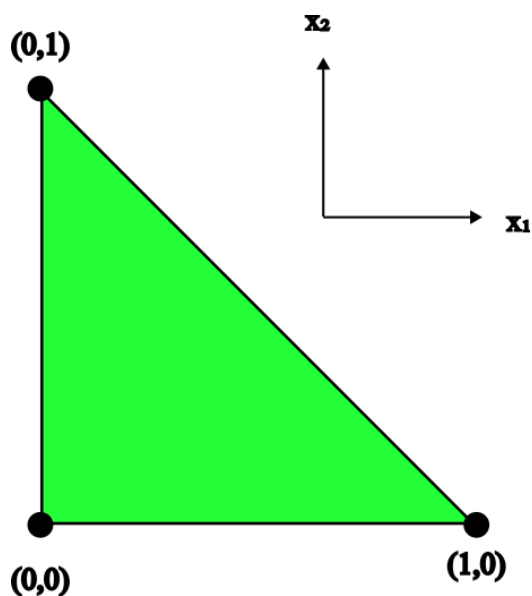


Figure 8: A sketch of a finite element reference triangle. Artistic inspiration from Farrell [7].

Appendix B: An ‘L-shaped’ domain

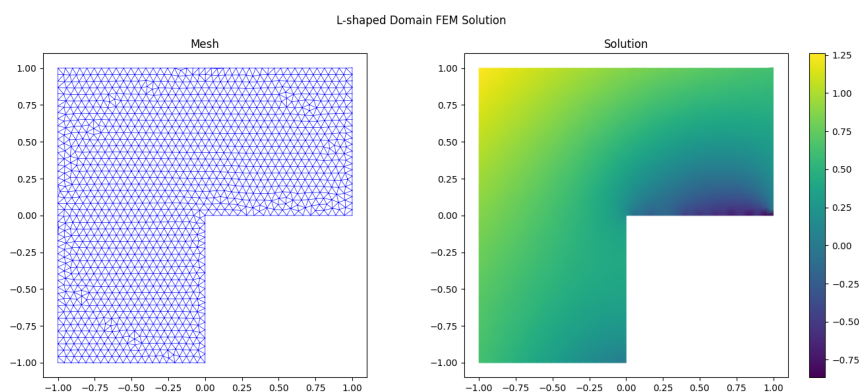


Figure 9: A mesh and solution surface in an L-shaped domain, to solve Laplace’s equation with homogeneous boundary conditions. We note that the mesh looks ‘correct’ visually, though as we can see from Figure 10, our algorithm does not converge, because of the central singularity of the solution.

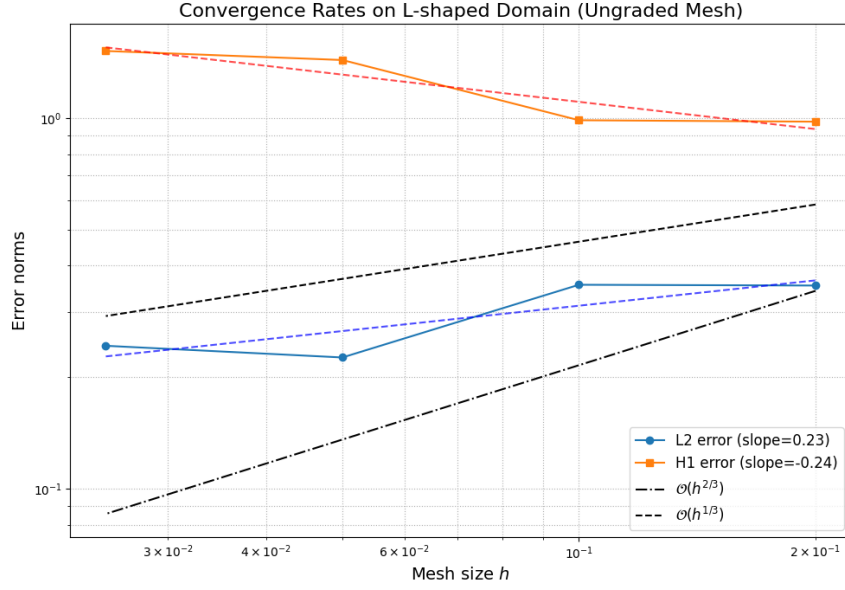


Figure 10: A plot showing the convergence (or lack thereof) of Laplace’s equation on an L-shaped domain with homogeneous Dirichlet boundary conditions, naïvely implementing the same algorithm for solving our problem via finite element methods as in the previous two cases, with vastly different results. As we can see in Figure 9, although the mesh seems visually uniform, this is not optimal for such cases.

Appendix C: Three-quarter circular domain

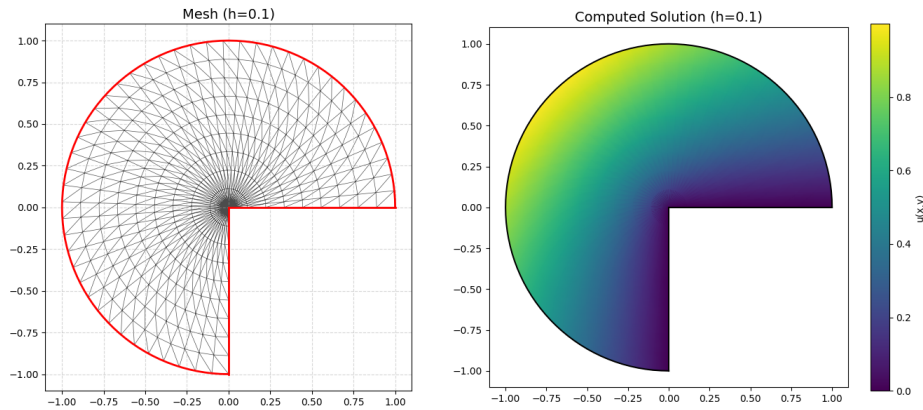


Figure 11: The mesh and computed solution for the three-quarter circular domain for Laplace’s equation with homogeneous boundary conditions with mesh size $h = 0.1$. We can see the cells converge singularly at the origin, in order to provide robust meshing and solution approximation around the singularity point.